

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Đặc tả Test Case và API Testing với Postman/Newman

Buổi 9 / 12 — Môn: Kiểm thử Phần mềm

Ngày 1 tháng 8 năm 2026

Nội dung buổi học

- 1 Vị trí trong đề cương — từ thiết kế đến thực thi test case
- 2 Khuôn mẫu đặc tả một ca kiểm thử chuẩn
- 3 REST API cơ bản và bản đồ API của DigiShield
- 4 Postman: collection, request đầu tiên, đăng nhập
- 5 Environment, variables và import OpenAPI
- 6 Scripts: `pm.test` / `pm.expect`
- 7 Data chaining — chuỗi nghiệp vụ SOC thật
- 8 Negative testing có hệ thống (401/403, body thiếu trường, biên)
- 9 Newman CLI: reporter, data-driven, tích hợp script
- 10 Thực hành trên lớp và bài nộp (gắn với BTL)

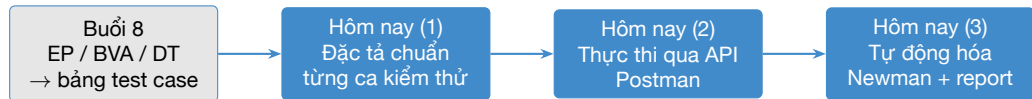
Vị trí trong đề cương môn học

Chương 5 — Kiểm thử hộp đen (buổi 8 và buổi 9)

- **Buổi 8:** các kỹ thuật thiết kế — EP, BVA, bảng quyết định, chuyển trạng thái, use case → ra được *bảng test case*.
- **Buổi 9 (hôm nay):** khuôn mẫu *đặc tả* một ca kiểm thử + *thực thi* hộp đen qua API bằng Postman/Newman.

Buổi	Chương	Chủ đề
8	C5 Hộp đen	EP, BVA, bảng quyết định, chuyển trạng thái
9	C5 Hộp đen	Đặc tả test case; Postman + Newman
10–11	C6 Selenium	Kiểm thử giao diện web E2E
12	C7 Thực tiễn	Regression, CI/CD, bảo vệ BTL

Từ bảng test case đến test tự động



Buổi 12: đưa vào CI/CD

Vấn đề của buổi 8

Bảng quyết định cho `InterceptionServiceImpl.evaluate()` mới chỉ là **thiết kế trên giấy**. Muốn giao cho người khác chạy được, lặp lại được, và tự động hóa được — cần **đặc tả chuẩn** và **công cụ thực thi**.

Vì sao cần khuôn mẫu đặc tả chuẩn?

Không có đặc tả chuẩn

- Test case “ở trong đầu” một người
- Người khác chạy → kết quả khác nhau
- Không truy vết được về yêu cầu
- Không đo được tiến độ kiểm thử

Có đặc tả chuẩn

- Bất kỳ ai cũng thực thi được y hệt
- Pass/Fail là khách quan
- Truy vết: yêu cầu ↔ test ↔ defect
- Là đầu vào để **tự động hóa**

Chuẩn tham chiếu

ISO/IEC/IEEE 29119-3 (Test documentation) định nghĩa các thành phần tối thiểu của một Test Case Specification.

Các trường của khuôn mẫu đặc tả

Trường	Nội dung
TC-ID	Mã duy nhất, vd TC-INT-001 (INT = module interception)
Tiêu đề	Một câu mô tả điều được kiểm thử
Mục tiêu	Kiểm chứng điều gì, vì sao quan trọng
Tham chiếu	ID yêu cầu / user story / dòng bảng quyết định
Kỹ thuật	EP / BVA / Decision Table / State / Use Case
Precondition	Trạng thái hệ thống + dữ liệu cần có trước khi chạy
Test Data	Dữ liệu đầu vào cụ thể (không ghi “giá trị hợp lệ”)
Test Steps	Các bước đánh số, mỗi bước một hành động
Expected Result	Kết quả mong đợi — cụ thể, đo được
Actual Result	Kết quả thực tế (điền khi thực thi)
Status	Pass / Fail / Blocked / Not Run
Severity	Mức nghiêm trọng nếu fail (Critical → Low)
Người thực hiện, ngày	Ai chạy, chạy lúc nào, trên môi trường nào

Nguyên tắc viết Expected Result

Expected Result phải: cụ thể — đo được — không mơ hồ

Sai — quá mơ hồ

- “Hệ thống chặn giao dịch”
- “Trả về lỗi”
- “API hoạt động đúng”

Đúng — cụ thể

- “HTTP 200;
decision="PAUSE"”
- “HTTP 403; không lộ dữ liệu”
- “signals chứa đúng 3 phần tử”

Quy tắc vàng

Nếu hai người đọc cùng đặc tả mà kết luận Pass/Fail khác nhau thì Expected Result **chưa đủ cụ thể**.

Ví dụ hoàn chỉnh: TC-INT-001 (1/2)

TC-INT-001 — Giao dịch tới tài khoản watchlist khi đang nghe điện thoại

TC-ID	TC-INT-001
Tiêu đề	Chuyển tiền tới tài khoản trong watchlist khi đang nghe điện thoại và người nhận mới → PAUSE
Mục tiêu	Kiểm chứng quy tắc can thiệp cao nhất của module interception (chống lừa đảo cuộc gọi)
Tham chiếu	REQ-INT-05; bảng quyết định buổi 8, dòng R1 (onCall = T, newPayee = T, watchlistHit = T)
Kỹ thuật	Decision Table
Ưu tiên / Severity	Cao / Critical
Precondition	Backend dev đang chạy; có token vai ANALYST (POST /auth/login); tài khoản 0123456789 đã có trong watchlist (POST /account-watchlist)
Test Data	userId = UUID demo; amount = 25000000; destAccount = "0123456789"; onCall = true; newPayee = true

Ví dụ hoàn chỉnh: TC-INT-001 (2/2)

TC-INT-001 — các bước và kết quả mong đợi

Test Steps	1. Gửi POST /api/v1/interventions/ evaluate với body JSON ở Test Data 2. Ghi nhận HTTP status 3. Ghi nhận body: decision, signals, message
Expected	HTTP 200; decision = "PAUSE"; signals chứa ON_CALL, NEW_PAYEE, WATCHLIST_HIT; message là chuỗi cảnh báo khác rỗng; một bản ghi InterventionEvent được lưu
Actual	(điền khi thực thi)
Status	Pass / Fail / Blocked / Not Run
Người thực hiện	(tên SV) — môi trường dev H2, ngày chạy

Nhận xét

Đặc tả này đủ chi tiết để: (1) người khác chạy tay bằng Postman, (2) chuyển thẳng thành test script tự động — ta sẽ làm cả hai hôm nay.

Trạng thái thực thi và Severity

Status của test case

Not Run	Chưa thực thi
Pass	Actual = Expected
Fail	Khác Expected → log defect
Blocked	Không chạy được (precondition fail)

Severity khi fail

Critical	Mất tiền, lộ dữ liệu, sập hệ thống
High	Nghiệp vụ chính sai
Medium	Sai ở luồng phụ
Low	Cosmetic, thông báo

Ví dụ DigiShield

TC-INT-001 fail (không PAUSE giao dịch lừa đảo) = **Critical**; thông báo message sai chính tả = **Low**. Severity quyết định thứ tự sửa lỗi, không phải thứ tự phát hiện.

Quản lý test case: công cụ

Công cụ	Ưu điểm	Hạn chế
Excel / Google Sheets	Miễn phí, quen thuộc, đủ cho BTL	Không truy vết tự động, dễ lệch version
TestRail	Quản lý run, báo cáo, milestone	Trả phí
Xray / Zephyr (Jira)	Gắn thẳng vào issue, truy vết yêu cầu-test-bug	Cần hệ sinh thái Jira

Yêu cầu cho BTL

Nhóm dùng **Google Sheets** theo đúng khuôn mẫu 13 trường ở trên; mỗi test case một dòng; cột Status cập nhật sau mỗi vòng chạy.

Ma trận truy vết yêu cầu (RTM)

Requirement Traceability Matrix

Bảng hai chiều nối **yêu cầu** với **test case** — trả lời câu hỏi: yêu cầu nào chưa có test? test nào không phục vụ yêu cầu nào?

Yêu cầu	TC-INT-001	TC-INT-002	TC-INT-003	TC-INT-010
REQ-INT-05 (PAUSE)	×			
REQ-INT-06 (WARN)		×		
REQ-INT-07 (ALLOW)			×	
REQ-INT-09 (phân trang)				×

Liên hệ

Trường *Tham chiếu* trong khuôn mẫu chính là dữ liệu để dựng RTM. Điền nghiêm túc ngay từ đầu, cuối kỳ không phải “truy ngược”.

API testing = kiểm thử hộp đen ở tầng HTTP

Vì sao thực thi test case qua API?

- Đúng tinh thần hộp đen: chỉ thấy request vào / response ra.
- Nhanh và ổn định hơn test qua giao diện (buổi 10–11).
- Kiểm tra được routing, phân quyền, serialization JSON — những thứ unit test (buổi 5–7) không chạm tới.

Khía cạnh	Kỹ thuật	Ví dụ DigiShield	Assertion
Status code	EP / BVA	200, 401, 403	<code>status === 200</code>
Body	Cấu trúc, miền giá trị	decision, confidence	<code>oneOf, within</code>
Phân quyền	Ma trận role	LEARNER gọi API ANALYST	<code>status === 403</code>
Thời gian	Hiệu năng	< 500ms	<code>responseTime < 500</code>

Cấu trúc một HTTP request

4 thành phần

- **Method:** GET, POST, PATCH, DELETE
- **URL:** host + path + query
- **Headers:** Content-Type, Authorization, X-Tenant-Id,...
- **Body:** JSON

Request thật của TC-INT-001

```
POST http://localhost:8080
      /api/v1/interventions/evaluate
Content-Type: application/json
Authorization: Bearer {{
    access_token}}

{
  "userId": "6f1a...c2",
  "amount": 25000000,
  "destAccount": "0123456789",
  "onCall": true,
  "newPayee": true
}
```

HTTP status code cần thuộc

Thành công (2xx)

200 OK	GET/POST thành công
201 Created	Tạo tài nguyên (POST /account-watchlist)
202 Accepted	Nhận việc, xử lý sau (POST /auth/forgot-password)

Lỗi (4xx / 5xx)

400	Input không hợp lệ
401	Chưa xác thực
403	Sai quyền (role)
404	Không có tài nguyên
500	Lỗi phía server

Nguyên tắc: mỗi endpoint phải được test với **cả** nhóm 2xx (happy path) **và** nhóm 4xx/5xx (negative) — đây là EP áp lên status code.

Bản đồ API DigiShield

Base URL (profile dev)

`http://localhost:8080/api/v1` — spec đầy đủ:
`docs/DigiShield_openapi.yaml`

Nhóm	Endpoint tiêu biểu	Quyền
Auth	POST /auth/login, /auth/refresh; GET /auth/me	login: công khai
Interception	POST /interventions/evaluate; GET /interventions?page&size; GET/POST /account-watchlist; GET /account-watchlist/check?value=	đã đăng nhập / ANALYST
Reporting	POST /reports/phishing; GET /reports/phishing?status=; POST /reports/phishing/{id}/triage	LEARNER / ANALYST
AI	POST /ai/classify, /ai/moderate; GET/POST /ai/templates; POST /ai/orchestration/run	ANALYST / CONTENT_EDITOR / ORG_ADMIN
Simulation	GET/POST /sim/campaigns; POST /sim/events	theo role
Analytics	GET /analytics/dashboard, /risk, /benchmark	theo role
Users	GET/POST /users; POST /users/import	quản trị

Xác thực ở profile dev

Xác thực khi test ở dev

- 1 DevSecurityConfig: permitAll
— không cần token.
- 2 POST /auth/login → TokenPair;
dev dùng StubAuthProvider —
password bị bỏ qua.

Các role

SUPER_ADMIN, ORG_ADMIN, MANAGER,
CONTENT_EDITOR, ANALYST, LEARNER (+
TENANT_ADMIN — alias cũ)

Chú ý

Ma trận role = “đắt” cho **test tiêu cực phân quyền** (LEARNER gọi API ANALYST
→ 403) — @PreAuthorize chỉ bật *ngoài* dev.

Tài khoản demo

```
admin@coquan.gov.vn  
analyst@coquan.gov.vn  
editor@coquan.gov.vn  
learner@coquan.gov.vn  
manager@coquan.gov.vn  
superadmin@digishield.vn  
# tenant demo: 11111111-  
# 1111-1111-1111-111111111111
```

Khởi động backend và smoke check

Bước 1: chạy backend (không cần Docker)

```
cd digishield
./gradlew :boot:app:bootRun \
  --args='--spring.profiles.active=dev'
# H2 in-memory + du lieu demo tu seed
```

Bước 2: smoke check bằng curl

```
curl http://localhost:8080/actuator/health
# {"status":"UP"}

curl http://localhost:8080/api/v1/reports/phishing
# JSON danh sach bao cao phishing demo
# (dev: khong can token, tenant mac dinh demo)
```

Postman — các thành phần cốt lõi

Khái niệm

- **Workspace:** không gian làm việc nhóm
- **Collection:** nhóm request liên quan
- **Folder:** phân cấp trong collection
- **Request:** một HTTP call
- **Environment:** tập biến theo môi trường
- **Scripts:** JS chạy trước/sau request

Luồng làm việc buổi hôm nay

- 1 Tạo Environment dev-local
- 2 Import OpenAPI → collection
- 3 Request đầu tiên + Tests
- 4 Data chaining chuỗi SOC
- 5 Negative tests
- 6 Export → chạy Newman

Request đầu tiên: health check (1/2)

GET {{base_url}}/actuator/health

Method: GET

URL: {{base_url}}/actuator/health

Tab *Scripts / Post-response* — 2 test đầu tiên

```
pm.test("Status 200", function () {  
    pm.response.to.have.status(200);  
});  
  
pm.test("Hệ thống đang UP", function () {  
    const body = pm.response.json();  
    pm.expect(body.status).to.equal("UP");  
});
```

Request đầu tiên: health check (2/2)

Ý nghĩa

Đặt request này **đầu collection**: nếu fail thì mọi request sau Blocked — đúng vai trò *precondition* trong đặc tả test case.

Đăng nhập: POST /auth/login (1/2)

Request

```
Method: POST
URL:      {{base_url}}/api/v1/auth/login
Header: Content-Type: application/json
Body:
{
  "email":      "analyst@coquan.gov.vn",
  "password":   "demo1234",
  "role":       "ANALYST"
}
```

Đăng nhập: POST /auth/login (2/2)

Response: TokenPair

```
{  
  "access_token": "dev-access-...",  
  "refresh_token": "dev-refresh-...",  
  "expires_in": 3600  
}
```

Tests cho /auth/login

```
pm.test("Status 200", () =>
  pm.response.to.have.status(200));

const body = pm.response.json();

pm.test("Có access_token và refresh_token", () => {
  pm.expect(body).to.have.property("access_token");
  pm.expect(body).to.have.property("refresh_token");
});

pm.test("expires_in là số dương", () => {
  pm.expect(body.expires_in).to.be.a("number");
  pm.expect(body.expires_in).to.be.above(0);
});

// Lưu token cho các request sau (data chaining)
pm.environment.set("access_token", body.access_token);
```


Các tab của một request trong Postman

Tab	Dùng để
Params	Query string, vd ?page=1&size=20
Authorization	Kiểu auth (Bearer token) áp cho request/folder
Headers	Content-Type, X-Tenant-Id
Body	raw JSON gửi đi
Scripts / Pre-request	JS chạy trước khi gửi
Scripts / Post-response (Tests)	JS assertion chạy sau khi nhận

Mẹo cấp collection

Token Bearer khai báo một lần ở cấp **folder** (tab Authorization) thay vì lặp lại từng request — các request con kế thừa.

Environment dev-local

Các biến khởi tạo

Environment: dev-local

```
-----  
base_url      = http://localhost:8080  
tenant_id     = 11111111-1111-1111-  
               1111-111111111111  
access_token  = (set bởi Tests của login)  
user_id       = (set bởi Tests của /auth/me)  
report_id     = (set bởi chuỗi reporting)
```

Cú pháp dùng biến

{{base_url}}/api/v1/auth/me — Postman thay giá trị khi gửi. Đổi môi trường (dev → staging) mà không sửa request.

Phân loại biến trong Postman

Loại	Phạm vi	Cú pháp set
Global	Toàn workspace	<code>pm.globals.set()</code>
Environment	Một environment	<code>pm.environment.set()</code>
Collection	Một collection	<code>pm.collectionVariables.set()</code>
Local	Một request	<code>pm.variables.set()</code>
Data	Một iteration (CSV/JSON)	từ file, đọc bằng <code>pm.iterationData</code>

Độ ưu tiên khi trùng tên (cao → thấp): Local > Data > Environment > Collection > Global.

Quy ước buổi này: token, id sinh ra khi chạy → **Environment**; bộ dữ liệu BVA → **Data file** (phần Newman).

Import OpenAPI: sinh collection tự động

DigiShield có sẵn spec máy đọc được

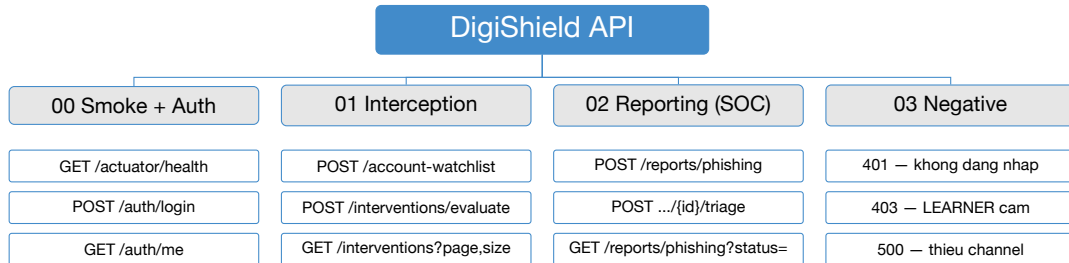
File docs/DigiShield_openapi.yaml — cũng chính là file frontend dùng để sinh client (Orval). Một nguồn sự thật cho cả dev và test.

- 1 Postman → *Import* → chọn file DigiShield_openapi.yaml
- 2 Chọn *Generate collection from imported APIs*
- 3 Postman tạo collection với đầy đủ folder theo tag, request mẫu, schema body
- 4 Sửa biến baseUrl của collection thành {{base_url}}/api/v1

Lưu ý

Collection sinh tự động **chưa có test script** — OpenAPI mô tả “API trông thế nào”, còn “đúng hay sai” vẫn là việc của tester.

Cấu trúc collection DigiShield API



Nguyên tắc tổ chức

Folder đánh số theo thứ tự chạy; mỗi request map về một TC-ID trong bảng đặc tả (ghi TC-ID vào phần mô tả request).

Pre-request vs Post-response

Pre-request script

Chạy **trước** khi gửi request:

- Sinh dữ liệu động: UUID, timestamp
- Tính giá trị phụ thuộc
- Chuẩn bị precondition

Post-response (Tests)

Chạy **sau** khi nhận response:

- Assertion Pass/Fail
- Lưu biến cho request sau
- Ghi log chẩn đoán

Đối chiếu với khuôn mẫu đặc tả

Pre-request $\approx$ *Precondition* + *Test Data*; Post-response $\approx$ *Expected Result*. Đặc tả tốt thì chuyển sang script gần như một-một.

Cấu trúc pm.test + pm.expect

Một assertion = một pm.test có tên

```
pm.test("Ten mo ta hanh vi mong doi", function () {  
    pm.expect(giaTriThucTe).to.equal(mongDoi);  
});
```

Object hay dùng

- pm.response.code
- pm.response.json()
- pm.response.responseTime
- pm.response.headers

Chai matcher hay dùng

- .to.equal / .to.eql
- .to.have.property
- .to.be.oneOf([...])
- .to.be.within(a, b)

Assertions cơ bản: status, time, header

// 1. Status code

```
pm.test("Status 200", function () {  
    pm.response.to.have.status(200);  
});
```

// 2. Thời gian phản hồi

```
pm.test("Phản hồi dưới 500ms", function () {  
    pm.expect(pm.response.responseTime)  
        .to.be.below(500);  
});
```

// 3. Header Content-Type

```
pm.test("Tra về JSON", function () {  
    pm.expect(pm.response.headers.get("Content-Type"))  
        .to.include("application/json");  
});
```


Ví dụ: POST /ai/classify — request

Phân loại nội dung nghi lừa đảo (module ai)

```
Method: POST
URL:      {{base_url}}/api/v1/ai/classify
Header: Content-Type: application/json
Header: Authorization: Bearer {{access_token}}
Body:
{
  "payload": "Tai khoan sap bi khoa!
             Xac minh OTP tai http://b1t.ly/xacminh"
}
```

Đặc tả response (từ OpenAPI)

`label` $\in$ {threat, spam, clean}; `confidence` $\in$ [0, 1]; `reason`: chuỗi giải thích. Dev dùng `StubAiClient` (heuristic) — kết quả **xác định**, test được.

Ví dụ: POST /ai/classify — tests

```
pm.test("Status 200", () =>
  pm.response.to.have.status(200));

const body = pm.response.json();

// Mien gia tri roi rac -> oneOf
pm.test("label thuoc {threat, spam, clean}", () =>
  pm.expect(body.label)
    .to.be.oneOf(["threat", "spam", "clean"]));

// Mien gia tri lien tuc -> within (BVA!)
pm.test("confidence trong [0, 1]", () => {
  pm.expect(body.confidence).to.be.a("number");
  pm.expect(body.confidence).to.be.within(0, 1);
});

pm.test("Co reason giai thich", () =>
  pm.expect(body.reason).to.be.a("string")
    .and.to.have.length.above(0));
```

Assertions nâng cao

```
const body = pm.response.json();

// id phải là UUID hợp lệ
pm.test("id dung dinh dang UUID", function () {
  const re = new RegExp("^[0-9a-f]{8}-[0-9a-f]{4}-"
    + "[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$", "i");
  pm.expect(body.id).to.match(re);
});

// Mọi phần tử danh sách dữ liệu bắt buộc
pm.test("Mọi report dữ liệu", function () {
  body.forEach(r => {
    pm.expect(r).to.include.keys(
      "id", "status", "aiLabel");
  });
});

// Không lộ thông tin nhạy cảm (security)
pm.test("Không lộ passwordHash", () =>
  pm.expect(body).to.not.have.property(
    "passwordHash"));
```

Data chaining — nguyên lý

Vấn đề

Nhiều test case có *precondition* là kết quả của request khác: triage cần `report_id`, mà id chỉ có sau khi submit report.



Ba bước kỹ thuật

- 1 Tests của request trước: `pm.environment.set("x", body.x)`
- 2 Request sau dùng `{{x}}` trong URL/body/header
- 3 Chạy **theo thứ tự** bằng Collection Runner / Newman

Chaining bước 1–2: token và user_id

Tests của POST /auth/login (đã viết ở trên)

```
pm.environment.set("access_token",  
  pm.response.json().access_token);
```

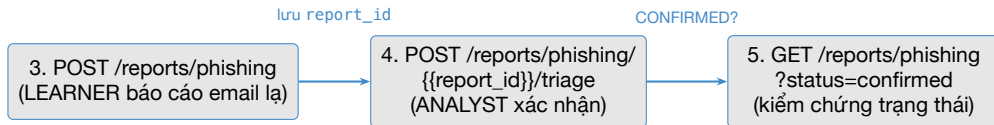
GET /auth/me — dùng token, lưu user_id

URL: {{base_url}}/api/v1/auth/me

Header: Authorization: Bearer {{access_token}}

```
pm.test("Status 200", () =>  
  pm.response.to.have.status(200));  
const me = pm.response.json();  
pm.test("Co id va email", () => {  
  pm.expect(me).to.have.property("id");  
  pm.expect(me.email).to.include("@");  
});  
pm.environment.set("user_id", me.id);
```

Chuỗi nghiệp vụ SOC thật: report → triage → verify



Đây là một use case test (buổi 8) chạy bằng máy

Luồng “người học báo cáo → chuyên viên SOC xác nhận mỗi đe dọa” — đúng kịch bản chính của module reporting; trạng thái chuyển SUBMITTED → CONFIRMED (state transition!).

Chaining bước 3: submit report

Request

```
Method: POST
URL:     {{base_url}}/api/v1/reports/phishing
Header: Authorization: Bearer {{access_token}}
Body:
{
  "userId": "{{user_id}}",
  "payload": "Email gia mao ngan hang,
             link la: http://b1t.ly/xacminh"
}
```

Tests — lưu report_id

```
pm.test("Status 200", () =>
  pm.response.to.have.status(200));
const r = pm.response.json();
pm.test("Trang thai ban dau SUBMITTED", () =>
  pm.expect(r.status).to.equal("SUBMITTED"));
pm.environment.set("report_id", r.id);
```

Chaining bước 4: triage xác nhận mối đe dọa

Request — đổi vai sang ANALYST

Method: POST

URL: {{base_url}}/api/v1/reports/phishing/
 {{report_id}}/triage

Header: Authorization: Bearer {{access_token}}

Body: { "confirmThreat": true }

Tests

```
pm.test("Status 200", () =>
  pm.response.to.have.status(200));
const r = pm.response.json();
pm.test("Chuyen sang CONFIRMED", () =>
  pm.expect(r.status).to.equal("CONFIRMED"));
pm.test("AI label la THREAT", () =>
  pm.expect(r.aiLabel).to.equal("THREAT"));
```


Chaining bước 5: kiểm chứng độc lập

GET /reports/phishing?status=confirmed (ANALYST)

```
pm.test("Status 200", () =>
  pm.response.to.have.status(200));

pm.test("Report có trong danh sách CONFIRMED", function () {
  const ids = pm.response.json().map(r => r.id);
  pm.expect(ids).to.include(
    pm.environment.get("report_id"));
});
```

Vì sao cần bước 5?

Bước 4 chỉ tin vào *response của chính nó*. Bước 5 đọc lại bằng endpoint khác → xác nhận trạng thái **thực sự được lưu** — tương tự nguyên tắc “verify qua kênh độc lập” của integration test buổi 7.

Collection Runner — chạy cả chuỗi

Cách chạy

- 1 Chọn collection → *Run collection*
- 2 Chọn Environment `dev-local`
- 3 Iterations = 1, giữ nguyên thứ tự
- 4 *Run DigiShield API*

Thứ tự là sống còn

Login fail → `access_token` rỗng → các request sau fail dây chuyền. Trong biên bản test, chúng ghi là **Blocked**, không phải Fail.

Đọc kết quả

Mỗi `pm.test` hiện một dòng xanh (pass) / đỏ (fail) kèm tên — vì vậy **tên test phải mô tả hành vi**, không đặt “test 1”, “test 2”.

Negative testing — 4 nhóm cần phủ

Nhóm	Ví dụ DigiShield	Expected
Xác thực (authn)	Gọi API không có token	401 Unauthorized
Phân quyền (authz)	LEARNER gọi GET /interventions	403 Forbidden
Body thiếu/sai trường	POST /ai/templates thiếu channel	4xx + thông báo lỗi
Giá trị biên (BVA)	size = 0 / 101 khi phân trang	theo đặc tả

Nguyên tắc

Negative test không phải “thử phá cho vui”: mỗi ca phải xuất phát từ **EP/BVA buổi 8** hoặc **ma trận phân quyền**, và có TC-ID riêng.

Ma trận role × endpoint

Endpoint	Không auth	LEARNER	ANALYST
GET /interventions	401	403	200
POST /account-watchlist	401	403	201
POST /ai/classify	401	403	200
POST /reports/phishing	401	200	403*
POST .../{id}/triage	401	403	200

*: endpoint submit yêu cầu role LEARNER — chiều “cấm ngược” cũng cần test.

Cách dùng

Mỗi ô của ma trận = 1 test case. Với 5 endpoint × 3 ngữ cảnh = 15 TC phân quyền, viết một lần, chạy lại mãi mãi (regression).

Test 401 và 403

TC-SEC-001: không đăng nhập → 401

```
// GET {{base_url}}/api/v1/interventions
// KHONG gui token (chay tren profile prod-like)
pm.test("Tu choi khi chua xac thuc: 401", () =>
  pm.response.to.have.status(401));
```

TC-SEC-002: LEARNER gọi API của ANALYST → 403

```
// GET {{base_url}}/api/v1/interventions
// Header: Authorization: Bearer <token LEARNER>
pm.test("Sai quyen: 403 Forbidden", () =>
  pm.response.to.have.status(403));

pm.test("Khong lo du lieu can thiep", () =>
  pm.expect(pm.response.text())
    .to.not.include("decision"));
```

Body thiếu trường — và một defect thật (1/2)

TC-AI-005: POST /ai/templates thiếu channel

Header: Authorization: Bearer {{editor_token}}

Body: { "subject": "Uu dai 0% phi chuyen khoan" }
// khong co "channel"

```
pm.test("Input thieu truong -> 400", () =>  
  pm.response.to.have.status(400));
```

Body thiếu trường — và một defect thật (2/2)

Chạy thật: test FAIL — đây là điều tốt!

DigiShield validate thủ công trong service
(`IllegalArgumentException("channel is required")`), không có handler ánh xạ → server trả **500** thay vì 400. Expected (theo đặc tả) $\neq$ Actual → ghi defect DEF-AI-01, severity Medium, đính kèm TC-AI-005.

Bài học

Negative test tồn tại để tìm đúng loại lỗi này: hệ thống “có chặn” nhưng chặn *sai cách* (500 làm client không phân biệt được lỗi gì).

Giá trị biên buổi 8: phân trang size

Hành vi thật của `listInterventions(page, size)`

```
// InterceptionServiceImpl (rút gọn)
int safePage = Math.max(page, 1);
int safeSize = Math.min(Math.max(size, 1), 100);
```

size gửi lên	0	1	2	101
Hành vi	clamp về 1	giữ 1	giữ 2	clamp về 100
Status	200	200	200	200

Câu hỏi thảo luận

Server *clamp* thay vì trả 400. Đặc tả không nói gì → tester phải **hỏi và chốt** với dev/BA, rồi ghi quyết định vào trường Expected của TC. Không tự đoán — “ambiguity” là kẻ thù số một của Expected Result.

Newman — Postman không cần chuột

Newman là gì?

Command-line runner cho Postman collection: chạy headless, trả **exit code**, xuất report — nền tảng để tự động hóa và đưa vào CI/CD (buổi 12).

Vì sao cần?

- Chạy toàn bộ regression một lệnh
- Không phụ thuộc GUI, chạy được trên server
- Kết quả máy đọc được (JSON/exit code)

Cài đặt (cần Node.js)

```
npm install -g newman
```

```
npm install -g \
  newman-reporter-htmlextra
```

```
newman --version
```

Newman — chạy collection DigiShield

Export từ Postman rồi chạy

```
# Export: Collection v2.1 + Environment
newman run digishield.postman_collection.json \
  -e dev-local.postman_environment.json
```

Output mẫu (rút gọn)

```
DigiShield API
-> 00 Smoke + Auth
  GET /actuator/health      [200, 45ms]
  POST /auth/login          [200, 82ms]
-> 02 Reporting (SOC)
  POST /reports/phishing    [200, 120ms]
  POST .../triage           [200, 95ms]

assertions: 34 | failed: 1   # DEF-AI-01!
```

Exit code

0 = tất cả pass; 1 = có fail — script/CI dựa vào đây.

Newman reporters

Ba reporter dùng trong môn học

```
newman run digishield.postman_collection.json \
  -e dev-local.postman_environment.json \
  --reporters cli,json,htmlextra \
  --reporter-json-export out/result.json \
  --reporter-htmlextra-export out/report.html \
  --reporter-htmlextra-title "DigiShield API"
```

cli	In terminal — dùng khi chạy tay
json	Máy đọc — parse thống kê, gửi Slack/Jira
htmlextra	Report HTML đẹp: pass rate, request/response từng ca, biểu đồ thời gian — file nộp bài của BTL

Data-driven testing với Newman

Ý tưởng: 1 request + n bộ dữ liệu = n test case

Tách dữ liệu BVA ra file CSV; Newman lặp request theo từng dòng (mỗi dòng = 1 iteration).

bva-size.csv — từ bảng BVA buổi 8

```
size,max_items
0,1
1,1
2,2
100,100
101,100
```

Request dùng biến data

```
GET {{base_url}}/api/v1/interventions
    ?page=1&size={{size}}
Header: Authorization: Bearer {{access_token}}
```

Data-driven: assertion động + lệnh chạy

Tests đọc pm.iterationData

```
const size = pm.iterationData.get("size");
const max  = parseInt(
  pm.iterationData.get("max_items"));

pm.test(`size=${size} -> status 200`, () =>
  pm.response.to.have.status(200));

pm.test(`size=${size} -> tra ve toi da ${max}`,
  function () {
    pm.expect(pm.response.json().length)
      .to.be.at.most(max);
  });
```

Chạy 5 iteration, chỉ folder BVA

```
newman run digishield.postman_collection.json \
  -e dev-local.postman_environment.json \
  -d bva-size.csv --folder "04 BVA size"
# iteration 1..5, moi dong CSV mot lan
```

Đóng gói vào script của BTL

package.json trong thư mục test của nhóm

```
{
  "scripts": {
    "test:api": "newman run postman/digishield.postman_collection.json -e postman/dev-local.postman_environment.json --reporters cli,htmlextra --reporter-htmlextra-export reports/api-report.html"
  }
}
```

Một lệnh chạy toàn bộ regression API

```
npm run test:api && echo "API OK"
# exit code != 0 -> regression fail
```

Nhìn trước buổi 12

Repo DigiShield có sẵn bản tương tự tại `digishield/postman/` (chạy `npm run test:api` từ thư mục đó). Đúng lệnh này được đặt vào GitHub Actions: mỗi lần push code, bộ API test tự chạy — *continuous testing*.

Thực hành trên lớp — tổng quan

BT	Nội dung	Kỹ thuật	Thời gian
1	Environment + import OpenAPI	biến, collection	10 ph
2	Smoke + Auth: health, login, me	assertions, chaining	15 ph
3	Interception: watchlist + evaluate 3 ca (PAUSE/WARN/ALLOW)	bảng quyết định → API	20 ph
4	Chuỗi SOC: submit → triage → verify	data chaining	15 ph
5	Negative: 401, 403, thiếu channel, size=101	mã trận quyền, BVA	15 ph
6	Newman + report HTML	CLI, htmlextra	10 ph
Tổng			85 ph

Yêu cầu tối thiểu của collection

Collection DigiShield API nộp cuối giờ

- ≥ 8 **request**, mỗi request ≥ 3 assertion có tên mô tả
- Phủ **happy path**: health, login, me, evaluate (PAUSE), watchlist, classify, submit report, triage
- ≥ 4 **ca negative**: 401 (không auth), 403 (LEARNER gọi / interventions), body thiếu channel (ghi nhận defect), size=101 (clamp)
- ≥ 1 **chuỗi data chaining** hoàn chỉnh (report_id đi qua 3 request)
- Mỗi request ghi **TC-ID** tương ứng trong phần mô tả

Bài tập 3 gợi ý — 3 ca từ bảng quyết định

TC	Test data	Expected
TC-INT-001	onCall=T, newPayee=T, tài khoản trong watchlist	PAUSE
TC-INT-002	onCall=F, newPayee=F, tài khoản trong watchlist	WARN
TC-INT-003	tài khoản sạch	ALLOW

Bài tập 6 — chạy Newman, đọc report

Các bước

```
# 1. Export collection + environment tu Postman
# 2. Chạy Newman với reporter htmlextra
newman run digishield.postman_collection.json \
  -e dev-local.postman_environment.json \
  --reporters cli,htmlextra \
  --reporter-htmlextra-export report.html

# 3. Mở report.html trong trình duyệt
```

Checklist đọc report

- Pass rate bao nhiêu? Có nào fail — Fail thật hay Blocked?
- Có fail TC-AI-005 có đúng như defect DEF-AI-01 dự đoán?
- Request nào chậm nhất? Có vượt ngưỡng 500ms không?

Bài nộp (về nhà — gắn với BTL, hạn: trước buổi 10)

Sản phẩm nộp (theo nhóm BTL, cho module nhóm phụ trách)

- 1 **Bảng đặc tả test case** (Google Sheets): ≥ 10 TC theo khuôn mẫu 13 trường, có tham chiếu kỹ thuật buổi 8
- 2 `module.postman_collection.json`: ≥ 12 request (≥ 4 negative), mỗi request ≥ 3 assertion
- 3 `dev-local.postman_environment.json` + file CSV data-driven (nếu có)
- 4 `report.html` của Newman (htmlextra), pass rate $\geq 90\%$
- 5 **Defect log**: mỗi ca fail một dòng — TC-ID, mô tả, expected/actual, severity

Tiêu chí chấm: đặc tả rõ ràng truy vết được (30%) — assertion đúng và đủ (30%) — negative có hệ thống (20%) — Newman chạy được một lệnh, report sạch (20%).

Tóm tắt buổi 9 (1/2)

Đặc tả test case

- Khuôn mẫu 13 trường;
Expected phải đo được
- TC-INT-001: từ dòng R1 bảng quyết định thành đặc tả chạy được
- RTM: truy vết yêu cầu — test — defect

Postman

- Collection, environment, biến `{{ten_bien}}`, import OpenAPI
- `pm.test` / `pm.expect`; chaining token, `report_id`

Tóm tắt buổi 9 (2/2)

Negative + Newman

- Ma trận role × endpoint (401/403)
- Thiếu trường → 500: defect thật DEF-AI-01
- Biên size: clamp — phải chốt đặc tả
- Newman: exit code, htmlextra, -d CSV, `npm run test:api`

Buổi 10

Selenium WebDriver: kiểm thử qua giao diện web — tầng trên cùng của testing pyramid.

Tài liệu tham khảo

- [1] Slide bài giảng điện tử.
- [2] Paul Ammann & Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers, *The Art of Software Testing*, 2nd ed., John Wiley & Sons, 2004.
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023 — <https://istqb.org/>
- [5] Postman Learning Center — <https://learning.postman.com/>
- [6] Newman & newman-reporter-htmlextra — <https://github.com/postmanlabs/newman>
- [7] DigiShield: docs/DigiShield_openapi.yaml và mã nguồn các module interception, reporting, ai.